



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 1 Laboratuvar Föyü

Web'in Temelleri: Ağdan HTTP'ye

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>

🕒 Tahmini süre:	2 saat
💻 Gereksinimler:	Terminal/kabuk erişimi (macOS/Linux: Terminal, Windows: PowerShell), internet bağlantısı, curl, dig (macOS/Linux) veya nslookup (Windows), Python 3.12+

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim 📣 Neden Önemli?

Tarayıcınıza www.example.com yazdığınızda, ekranda bir sayfa belirene kadar perde arkasında görünmez bir dizi adım işler: bir isim bir IP adresine çözümlenir, bir port üzerinden bağlantı kurulur, düz metin hâlinde bir istek gönderilir ve yine düz metin hâlinde bir cevap döner. YG II'de `QTcpSocket` ile bu adımlardan bazılarını elle kurmuştunuz; bugün aynı süreci gerçek bir web sunucusuna karşı, uçtan uca, kendi gözünüzle izleyeceksiniz — hiç kod yazmadan, yalnızca `curl`, `dig`/`nslookup` ve küçük bir Python socket betiğiyle.

1 • Ön-öğretim 🔄 Ön Bilgi

Bu föy Hafta 1 ders notunun dört ünitesinin tamamına dayanır: Ünite 1 (TCP/IP katmanları, IP adresi, port, DNS), Ünite 2 (istemci–sunucu, statik/dinamik içerik, ters vekil), Ünite 3 (üç katmanlı WebApp mimarisi, API, SPA/SSR) ve Ünite 4 (HTTP istek/cevap anatomisi). YG II'nin TCP soket haftasından `connectToHost(ip, port)`, `bind/listen`, `send/receive` deneyiminizi hazır varsayıyoruz; bu föy o deneyime IP+port'un ne anlama geldiğini ve gönderdiğiniz baytların aslında hangi protokolün (HTTP) metnini taşıdığını ekler.

Föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi tamamlayın (komutu çalıştırın ya da istenen gözlemi yapın), sonra kendi çıktınızı *Beklenen sonuç* kutusuyla karşılaştırarak doğru yolda olduğunuzu hemen doğrulayın. Adımlar ders notunun dört ünitesiyle aynı sırada ilerler; her bölümün başında ilgili üniteye bir atıf bulacaksınız. Ağ üzerinden gerçek sunuculara istek attığınız için gördüğünüz IP adresi, UUID ya da zaman damgası *Beklenen sonuç* kutusunda farklı olabilir — önemli olan gözlemin *türüdür*, birebir sayısal değer değil. Birkaç adımda bir gelen *Kontrol Noktası*, o ana kadar gözlemlediklerinizi bir arada toparlamanıza yarar.

1 TCP/IP Katmanları: Bir Alan Adı Nasıl IP+Port'a Dönüşür?

(bkz. Ders notu Hafta 1, Ünite 1 — TCP/IP Katmanları, IP Adresi, Port ve DNS.)

Adım 1: Bir alan adını IP adresine çözümleyin

? Ne yapacağız? Terminalde `www.example.com` alan adının hangi IP adresine çözümlendiğini `dig` (macOS/Linux) ya da `nslookup` (Windows) ile sorgulayacaksınız.

i Neden? Tarayıcınıza yazdığınız bir alan adı, ağda doğrudan kullanılamaz; bir TCP bağlantısı kurulmadan önce mutlaka bir IP adresine çevrilmesi gerekir. Bu adım, YG II'de `connectToHost` çağrısına elle verdiğiniz IP'yi bu derste kimin (DNS'in) bulduğunu somutlaştırır.

Aşağıdaki, bu sorgulamayı otomatikleştiren betik satırları:

```
1 # "dig +short" yalnızca çözümlenen IP adres(ler)ini basar (ANSWER bölümünün
2 # özeti); Windows'ta karşılığı "nslookup www.example.com" olup çıktı
3 # içindeki "Address:" satırı aynı IP'yi gösterir.
4 if ! command -v dig >/dev/null 2>&1; then
5     echo "HATA: 'dig' komutu bulunamadi. Windows'ta yerine 'nslookup ${ALAN_ADİ}'
6     calistirin." >&2
7 else
8     DIG_CIKTI=$(dig +short "${ALAN_ADİ}" A 2>&1)
9     DIG_DURUM=$?
10    if [ ${DIG_DURUM} -ne 0 ] || [ -z "${DIG_CIKTI}" ]; then
11        echo "HATA: '${ALAN_ADİ}' çözümlenemedi. Internet/DNS bağlantınızı kontrol edin
12        ." >&2
13    else
14        echo "${DIG_CIKTI}"
15    fi
16 fi
```

>_ Terminal

\$ dig +short www.example.com A

👁 Beklenen sonuç: Ekranda dört nokta ile ayrılmış dört sayıdan oluşan bir IPv4 adresi görünür (ör. 93.184.216.34). Sizin gördüğünüz adres farklı olabilir — `example.com` birden çok IP adresine sahip olabilir; önemli olan bir adresin görünmesidir.

🖥 Beklenen Çıktı

93.184.216.34

🖥 macOS / Linux

`dig` çoğu macOS/Linux kurulumunda hazır gelir; ek kurulum gerekmez.

🖥 Windows

`dig` genelde kurulu değildir; yerine `nslookup www.example.com` çalıştırın. Çıktının biçimi farklıdır ama `Address:` ile başlayan satır aynı IP adresini gösterir.

Adım 2: `curl -v` ile bağlanılan IP:port'u gözlemleyin

? Ne yapacağız? Aynı alan adına `curl -v` ile bir HTTPS isteği gönderip çıktıda `Connected to` ile başlayan satırı bulacaksınız.

i Neden? Bir önceki adımda DNS'in bulduğu IP adresi tek başına yeterli değildir; bağlantı aynı zamanda bir **port** üzerinden kurulur. `curl -v` çıktısındaki bu satır, tarayıcının/curl'un

Adım 2 (devamı)

fiilen hangi IP:port ikilisine bağlandığını — Ünite 1'in IP+port ikilisini — gözünüzün önüne serer.

Aşağıdaki, bu gözlemi otomatikleştiren betik satırları:

```
1 # curl -v çıktısının "Connected to ... port ..." satiri, tarayicinin/curl'un
2 # DNS'ten aldığı IP adresine hangi porttan (https için varsayılan 443)
3 # bağlandığını gösterir.
4 CURL_CIKTI=$(curl -sS -v -m 10 "https://${ALAN_ADİ}" -o /dev/null 2>&1)
5 CURL_DURUM=$?
6 if [ ${CURL_DURUM} -ne 0 ]; then
7     echo "HATA: https://${ALAN_ADİ} adresine bağlanılamadı (curl kodu: ${CURL_DURUM}).
8       Internet bağlantınızı kontrol edin." >&2
9 else
10    echo "${CURL_CIKTI}" | grep -i "Connected to"
11 fi
```

>_ Terminal

```
$ curl -v https://www.example.com -o /dev/null
```

👁 Beklenen sonuç: Çıktı satırları arasında Connected to www.example.com (IP_ADRESİ) port 443 biçiminde bir satır görünür. Port numarasının **443** olmasının nedeni https:// kullanmanızdır — HTTPS'in varsayılan sözleşme portu (Ünite 2'de derinleşecek).

🖥 Beklenen Çıktı

```
* Connected to www.example.com (93.184.216.34) port 443
```

💡 İpucu

Uzun curl -v çıktısında tek bir satırı ararken curl -v https://www.example.com -o /dev/null 2>&1 | grep -i "connected to" gibi bir filtre kullanmak, gözünüzü yormadan doğrudan aradığınız satırı bulmanızı sağlar.

✅ **Kontrol Noktası — devam etmeden doğrulayın**

Bir önceki iki adımda gördüğünüz IP adresinin dig/nslookup çıktısıyla curl -v'nin Connected to satırındaki IP ile (yaklaşık olarak) örtüştüğünü ve bağlantı portunun **443** olduğunu doğruladıysanız, IP+port ikilisini doğru okumuşsunuz demektir — bir sonraki bölüme geçebilirsiniz.

2 Web Sunucusu: Statik mi, Dinamik mi?

(bkz. Ders notu Hafta 1, Ünite 2 — Standart Web Sunucu Mimarisi.)

Adım 3: Statik ve dinamik bir uç noktayı ikişer kez isteyip farkı gözlemleyin

❓ Ne yapacağız? kod_ornekleri/hafta01/statik-dinamik-istek.sh betiğini çalıştıracaksınız. Betik önce statik bir sayfayı (https://example.com/) iki kez, sonra dinamik bir uç noktayı (https://httpbin.org/uuid) iki kez ister ve iki cevabı karşılaştırır.

Adım 3 (devamı)

i Neden? Statik içerik, sunucunun diskte hazır duran bir dosyayı olduğu gibi gönderdiği durumdur — her istekte aynı cevap gelir. **Dinamik içerik** ise sunucunun isteğe göre anında **ürettiği** içeriktir — aynı adres, farklı istekte farklı cevap verebilir. Bu ayrımı gözünüzle görmeden önce ezbere bilmek yeterli değildir; bu adım size iki ardışık isteğin gövdesini birebir karşılaştırma imkânı verir.

```

1 # example.com sunucudaki hazır bir HTML dosyasını olduğu gibi gönderir; iki
2 # ardışık istek birebir aynı gövdeyi dondurmeli.
3 ISTEK1=$(curl -sS -m 10 "${STATIK_URL}")
4 DURUM1=$?
5 ISTEK2=$(curl -sS -m 10 "${STATIK_URL}")
6 DURUM2=$?
7
8 if [ ${DURUM1} -ne 0 ] || [ ${DURUM2} -ne 0 ]; then
9     echo "HATA: ${STATIK_URL} adresine bağlanılamadı. Internet bağlantınızı kontrol
10     edin." >&2
11 elif [ "${ISTEK1}" = "${ISTEK2}" ]; then
12     echo "Statik cevap iki istekte de BIREBİR AYNI (beklenen davranış)."
```

```

13 else
14     echo "Statik cevap iki istekte farklı çıktı (beklenmiyordu)."
```

```

15 fi
16
17 # httpbin.org/uuid her istekte YENİDEN ÜRETİLEN bir rastgele kimlik
18 # dondurur; bu yüzden iki ardışık isteğin gövdesi FARKLI olmalıdır.
19 CEVAP1=$(curl -sS -m 10 "${DINAMIK_URL}")
20 DURUM3=$?
21 CEVAP2=$(curl -sS -m 10 "${DINAMIK_URL}")
22 DURUM4=$?
23
24 if [ ${DURUM3} -ne 0 ] || [ ${DURUM4} -ne 0 ]; then
25     echo "HATA: ${DINAMIK_URL} adresine bağlanılamadı. Internet bağlantınızı kontrol
26     edin." >&2
27 else
28     echo "1. cevap: ${CEVAP1}"
29     echo "2. cevap: ${CEVAP2}"
30     if [ "${CEVAP1}" != "${CEVAP2}" ]; then
31         echo "Dinamik cevap iki istekte FARKLI (beklenen davranış)."
```

```

32     else
33         echo "Dinamik cevap iki istekte aynı çıktı (beklenmiyordu – sunucu onbellek
34         olabilir)."
```

```

35     fi
36 fi
37
38 fi

```

>_ Terminal

```
$ bash kod_ornekleri/hafta01/statik-dinamik-istek.sh
```

👁 Beklenen sonuç: İlk bölümde "Statik cevap iki istekte de BIREBİR AYNI" mesajı görünür — sunucu diskteki aynı dosyayı gönderdi. İkinci bölümde iki farklı UUID değeri ile "Dinamik cevap iki istekte FARKLI" mesajı görünür — sunucu her istekte cevabı yeniden üretti.

Adım 3 (devamı)**Beklenen Çıktı**

== Statik istek: `https://example.com/` (iki kez) ==
Statik cevap iki istekte de BİREBİR AYNI (beklenen davranış).

== Dinamik istek: `https://httpbin.org/uuid` (iki kez) ==
1. cevap: `{"uuid": "3f2504e0-4f89-11d3-9a0c-0305e82c3301"}`
2. cevap: `{"uuid": "9c858901-8a57-4791-81fe-4c455b099bc9"}`
Dinamik cevap iki istekte FARKLI (beklenen davranış).

Ortak Hata / Tuzak

İki UUID'in *farklı* çıkması bir hata değil, tam olarak **beklenen** sonuçtur; `httpbin.org/uuid` her istekte kasıtlı olarak yeni bir kimlik üretir. Betiğin "farklı çıktı" demesinden şüphelenip yeniden çalıştırmanıza gerek yoktur.

3 WebApp Mimarisi: Üç Katmanı Yerleştirin

(bkz. Ders notu Hafta 1, Ünite 3 — WebApp Mimarisi: Üç Katman, API, SPA vs SSR.)

Adım 4: Etkinlik/Bilet Sisteminin üç katmanlı haritasını okuyun ve yeni cümleleri katmanlara yerleştirin

? Ne yapacağız? Aşağıdaki şemayı inceleyip Etkinlik/Bilet Sisteminin sunum, iş ve veri katmanlarının hangi teknolojiyle karşılandığını belirleyeceksiniz. Ardından şu üç cümleyi — kâğıda ya da zihninizde — sunum/iş/veri katmanlarından birine yerleştirin: (a) "Kullanıcı arayüzünde etkinlik kartları liste hâlinde gösterilir." (b) "Aynı kullanıcının aynı etkinliğe iki kez bilet almasını engelleyen kural uygulanır." (c) "Satılan bilet kaydı kalıcı olarak diskte saklanır."

i Neden? Üç katmanlı mimari (sunum/iş/veri) bu dersin geri kalanının iskeletidir; her hafta kodladığınız parça bu haritanın bir hücresine yerleşecek. Bu alıştırmayı, bir cümlemin *hangi* katmana ait olduğunu ayırt etme becerisini erkenden — kod yazmadan — kazandırır.

Etkinlik/Bilet Sistemi Mimarisi

👁 Beklenen sonuç: Şemada sunum katmanına **React**, iş katmanına **FastAPI**, veri katmanına **SQLite** yerleşir. Cümlelerin doğru yerleşimi: (a) **sunum katmanı** — yalnızca görsel gösterim, karar verilmiyor; (b) **iş katmanı** — bir kural/karar uygulanıyor; (c) **veri katmanı** — kalıcı saklama işi. Üçü de ayrı sorumluluklardır ve web mimarisinde genelde ayrı bileşenlerde yaşar.

✓ Kontrol Noktası — devam etmeden doğrulayın

Statik/dinamik farkını iki ardışık istekle gördüğünüzü ve üç cümleyi sunum/iş/veri katmanlarına doğru yerleştirdiğinizi doğruladıysanız, HTTP'nin kendi anatomisine geçebilirsiniz.

4 HTTP İstek/Cevap Anatomisi

(bkz. *Ders notu Hafta 1, Ünite 4 — HTTP İstek/Cevap Anatomisi.*)

Adım 5: Ham bir HTTP isteği/cevabını okuyun, sonra gerçek bir sunucuda aynı anatomiyi doğrulayın

❓ Ne yapacağız? Önce `kod_ornekleri/hafta01/ornek-istek.http` ve `kod_ornekleri/hafta01/ornek-cevap.http` dosyalarını açıp elle yazılmış istek/cevap metnindeki dört parçayı (istek/durum satırı, başlıklar, boş satır, gövde) işaretleyeceksiniz. Ardından `http-istek-anatomisi.sh` betiğini çalıştırıp aynı anatomiyi gerçek bir sunucuya (`httpbin.org`) karşı, `curl -v` ile bir kez daha göreceksiniz.

❗ Neden? HTTP'nin aslında TCP soket üzerinden gönderilen **düz metin** olduğunu önce elle yazılmış, sabit bir örnekte görmek; sonra aynı yapının gerçek bir sunucuda *birebir* tekrarlandığını görmek, Ünite 4'ün anatomi şemasını soyut bir çizimden çıkarıp somutlaştırmak. Ham istek metni (istek satırı + başlıklar + boş satır):

```
>_ Terminal
GET /etkinlikler HTTP/1.1
Host: biletsistemi.example.com
User-Agent: curl/8.4.0
Accept: application/json
```

Ham cevap metni (durum satırı + başlıklar + boş satır + gövde):

```
Beklenen Çıktı
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 52

{"etkinlik": "KTU Bahar Senligi", "kalanKoltuk": 42}
```

Şimdi aynı anatomiyi gerçek bir sunucuda doğrulayın:

```
1 # ">" ile başlayan satırlar giden isteği (istek satırı + başlıklar), "<"
2 # ile başlayan satırlar gelen cevabı (durum satırı + başlıklar) gösterir.
3 GET_CIKTI=$(curl -sS -v -m 10 --http1.1 "https://httpbin.org/get" 2>&1)
4 GET_DURUM=$?
5 if [ ${GET_DURUM} -ne 0 ]; then
6     echo "HATA: httpbin.org/get adresine baglanilamadi. Internet baglantinizi kontrol
7     edin." >&2
8 else
9     echo "${GET_CIKTI}" | grep -E "^(> |< )"
10 fi

1 # -d ile verilen JSON, istegin GOVDESINDE gonderilir; httpbin bu govdeyi
2 # "json" alaninda aynen geri yansitir – GET'in aksine POST'un govdesi doludur.
3 POST_CIKTI=$(curl -sS -m 10 -X POST "https://httpbin.org/post" \
4     -H "Content-Type: application/json" \
5     -d '{"ogrenci":"KTU","hafta":1}')
6 POST_DURUM=$?
7 if [ ${POST_DURUM} -ne 0 ]; then
8     echo "HATA: httpbin.org/post adresine baglanilamadi. Internet baglantinizi kontrol
9     edin." >&2
```

Adım 5 (devamı)

```

9 else
10     echo "${POST_CIKTI}"
11 fi

```

>_ Terminal

```
$ bash kod_ornekleri/hafta01/http-istek-anatomisi.sh
```

👁 **Beklenen sonuç:** > ile başlayan satırlar giden isteğinizi (istek satırı + başlıklar), < ile başlayan satırlar gelen cevabı (durum satırı + başlıklar) gösterir — tıpkı yukarıdaki ham istek/cevap metninizdeki gibi. POST örneğinde, gönderdiğiniz JSON gövdenin httpbin.org tarafından "json" alanında aynen yansıtıldığını görürsünüz — GET'in aksine POST'un gövdesi doludur.

🖥 Beklenen Çıktı

```

== GET istegi (curl -v ile giden/gelen basliklar) ==
> GET /get HTTP/1.1
> Host: httpbin.org
> User-Agent: curl/8.4.0
> Accept: */*
<
< HTTP/1.1 200 OK
< Content-Type: application/json
< Content-Length: 256

== POST istegi (JSON govde) ==
{
  "json": {
    "hafta": 1,
    "ogrenci": "KTU"
  },
  "url": "https://httpbin.org/post"
}

```

Adım 6: soket-istemci.py ile HTTP isteğini elle oluşturup gönderin

❓ **Ne yapacağız?** kod_ornekleri/hafta01/soket-istemci.py betiğini çalıştıracaksınız. Bu betik, curl'ün sizin için otomatik yaptığı işi — TCP bağlantısı açma, HTTP isteğini metin olarak oluşturup gönderme, ham cevabı okuma — Python'ın socket modülüyle elle yapar.

📌 **Neden?** Bu adım, haftanın en önemli kapanış fikrini kanıtlar: HTTP isteği aslında yalnızca belirli bir biçime uyan bir **dizedir** (*string*); curl ya da tarayıcınız bu dizeyi sizin için otomatik üretip bir TCP soketinden gönderir. YG II'de soket->write(...) ile bayt gönderdiğinizde, aslında — bunu o zaman bilmeseniz de — tam olarak bu haftanın öğrettiği HTTP metnini gönderiyor olabildiniz.

```

1 istek = (
2     f"GET / HTTP/1.1\r\n"
3     f"Host: {HOST}\r\n"
4     f"User-Agent: hafta01-soket-istemci\r\n"
5     f"Accept: text/html\r\n"
6     f"Connection: close\r\n"
7     f"\r\n"

```

Adım 6 (devamı)

```

8 )
9
10 try:
11     with socket.create_connection((HOST, PORT), timeout=10) as soket:
12         soket.sendall(istek.encode("ascii"))
13         cevap = b""
14         while True:
15             parca = soket.recv(4096)
16             if not parca:
17                 break
18             cevap += parca
19         print(cevap.decode("utf-8", errors="replace"))
20 except OSError as hata:
21     print(
22         f"HATA: {HOST}:{PORT} adresine baglanilamadi ({hata}). "
23         "Internet baglantinizi kontrol edin.",
24         file=sys.stderr,
25     )
26 sys.exit(1)

```

>_ Terminal

```
$ python3 kod_ornekleri/hafta01/soket-istemci.py
```

🕒 Beklenen sonuç: Ekranda HTTP/1.1 200 OK durum satırıyla başlayan, ardından başlıkları ve en sonunda HTML gövdesini içeren ham bir HTTP cevabı görünür — `curl -v`'nin bir önceki adımı gösterdiğiyle aynı anatomiye sahip, ama bu kez `curl` yerine sizin elle yazdığınız birkaç satır tarafından üretilmiş bir istek karşılığında gelmiştir.

🖥 Beklenen Çıktı

```

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked

<!doctype html>
<html>
<head>
  <title>Example Domain</title>
...

```

⚠ Dikkat

`soket-istemci.py` dosyasındaki istek metninin en sonunda **çift** `\r\n` bulunur (boş satır). Bu boş satır olmadan sunucu isteğin nerede bittiğini anlayamaz ve bağlantı zaman aşımına uğrar — Ünite 4'ün “boş satır, başlıklarla gövdeyi ayırır” kuralının canlı kanıtıdır.

✅ Kontrol Noktası — devam etmeden doğrulayın

Ham istek/cevap metnindeki dört parçayı (istek/durum satırı, başlıklar, boş satır, gövde) doğru işaretlediyseniz, `curl -v` çıktısında `>/<` ayrımını gördüyseniz ve `soket-istemci.py` ile HTTP/1.1 200 OK durum satırını aldıysanız, HTTP'nin düz metin doğasını doğru kavramışsınız.

nız demektir.

Adım 7: Kendiniz ekleyin: farklı bir durum kodu ailesini gözlemleyin

❓ Ne yapacağız? Şimdiye kadar yalnızca 2xx (başarılı) durum kodlarını gördünüz. `httpbin.org`'un `/status/<kod>` uç noktasını kullanarak **bir** 4xx (istemci hatası) ve **bir** 5xx (sunucu hatası) durum kodunu `curl -v` ile gözlemleyin (ör. `https://httpbin.org/status/404` ve `https://httpbin.org/status/500`).

i Neden? Ünite 4, durum kodlarını dört aileye ayırır: 2xx (başarılı), 3xx (yönlendirme), 4xx (istemci hatası), 5xx (sunucu hatası). Bu görev, şimdiye kadar hep 2xx gördüğünüz döngüyü kırıp aynı `curl -v` becerisini yeni bir duruma — bir hata cevabına — uygulamanızı sağlar.

>_ Terminal

```
$ curl -v https://httpbin.org/status/404 -o /dev/null
$ curl -v https://httpbin.org/status/500 -o /dev/null
```

👁 Beklenen sonuç: İlk komutta `< HTTP/1.1 404 NOT FOUND` satırını görürsünüz — istemcinin (sizin) var olmayan bir kaynak istediğiniz anlamına gelir. İkinci komutta `< HTTP/1.1 500 INTERNAL SERVER ERROR` satırını görürsünüz — bu kez hata sizde değil, sunucuda oluşmuştur.

💡 İpucu

`httpbin.org/status/<kod>` deseni, 100 ile 599 arasında hemen her durum kodunu isteyerek deneyebileceğiniz kullanışlı bir test uç noktasıdır (ör. `/status/301` ile bir yönlendirme — 3xx ailesi — de gözlemleyebilirsiniz).

Ortak Hata / Tuzak

4xx ile 5xx ailesini karıştırmayın: 4xx her zaman **isteği gönderenin** (sizin, ya da tarayıcının) bir şeyi yanlış yaptığı anlamına gelir (ör. olmayan bir yol istemek); 5xx ise hatanın **sunucu tarafında** olduğu anlamına gelir — siz doğru bir istek gönderseniz bile sunucu cevap üretirken çökebilir.

✖ Sorun Giderme

Belirti	Olası neden	Çözüm
dig komutu bulunamadı	Windows'ta dig genelde kurulu değildir	Yerine <code>nslookup www.example.com</code> çalıştırın
curl: (6) Could not resolve host	DNS/internet bağlantısı yok ya da alan adı yanlış yazıldı	İnternet bağlantınızı ve alan adının yazımını kontrol edin
curl -v çıktısında Connected to satırını bulamıyorum	Çıktı çok uzun, başlıklar arasında kayboldu	<code>curl -v ... 2>&1 grep -i "connected to"</code> ile filtreleyin
python3: command not found	Sisteminizde Python python adıyla kurulu	<code>python socket-istemci.py</code> deneyin
socket-istemci.py "hata: example.com:80 adresine bağlanılamadı" verdi	Kurumsal ağ/güvenlik duvarı 80 portunu engelliyor olabilir	Farklı bir ağdan (ör. mobil hotspot) yeniden deneyin
Betik "dinamik cevap aynı çıktı" uyarısı verdi	Nadiren <code>httpbin.org</code> kısa süreli önbellek uygulayabilir	Betiği birkaç saniye arayla yeniden çalıştırın

5 • İzleme ➡ Aktarım — Başka Nerede?

Bugün gördüğünüz dört beceri — IP+port çözümleme, statik/dinamik ayrımı, üç katmanlı harita, HTTP anatomisi — bu dönem boyunca Etkinlik/Bilet Sisteminde birer birer somutlaşacak. Hf.3'te sunum katmanının ilk statik HTML/CSS sayfasını yazdığınızda, bu haftanın "statik içerik" fikrini kendi elinizle üreteceksiniz. Hf.6'da FastAPI ile ilk API uç noktanızı yazdığınızda, bugün `httpbin.org/uuid`'de gördüğünüz "her istekte yeniden üretilen dinamik cevap" fikrini kendi kodunuzla kuracaksınız — ve `uvicorn app.main:app --port 8000` ile sunucunuzu ayağa kaldırdığınızda, `localhost:8000` yazınca tarayıcınızın yine bu haftanın öğrettiği IP+port çözümlemesini (bu kez yerel makinenizde) yaptığını fark edeceksiniz. Hf.8'de SQLAlchemy ile veri katmanını doldurduğunuzda, bugünkü üç katmanlı haritanın "veri" hücrelerini siz inşa etmiş olacaksınız. Hf.14'te ise bir nginx ters vekili kurarken, Ünite 2'de yalnızca kavram düzeyinde gördüğünüz "danışma görevlisi" fikrini gerçek bir yapılandırmaya dönüştüreceksiniz. Son olarak, `socket-istemci.py` ile bugün elle kurduğunuz istek — başlıklar + boş satır + gövde — Hf.6'dan itibaren FastAPI'nin sizin için otomatik ürettiği her cevabın altında, değişmeden yaşamaya devam edecek.